Corruption Robustness of AI-Generated Image Detectors: Augmentation, Pretraining and Generalisation to Unseen Degradations.

Katherine Pietroni
Department of Electrical Engineering and Computer Science
Queen Mary University of London
London, United Kingdom
k.pietroni@se25.qmul.ac.uk

William Mckie
Department of Electrical Engineering and Computer Science
Queen Mary University of London
London, United Kingdom
w.f.mckie@se25.qmul.ac.uk

Abstract. **Many models which have been trained to detect AI generated images are passed clean, undamaged images. However, real world images which genuinely need to be checked for synthetic generation have undergone degradation, recompression, rescaling and exhibit increased noise levels as a result of their passage via the internet. In this project, we address this discrepancy between standard training protocol and the real-world factors associated with AI image generation by asking whether corrupting images as a method of augmentation during training can help to build a detector which can generalise to unseen methods of corruption and whether ImageNet pretraining impacts the conclusion drawn. The CIFAKE dataset consists of 60,000 CIFAR-10 images and 60,000 fabricated images created with Stable Diffusion. We use this to train a 544k-parameter CNN as well as a pretrained ResNet-18, whereby each type of architecture gets trained using two different methods: training with clean images and training with both noise and JPEG augmentation. Three paired seeds are utilised per configuration, and Gaussian blur is not used during training so that the model's ability to generalise can be evaluated. This evaluation occurs over 16 different corruption conditions using 5 metrics. The use of augmented data in training increases the accuracy of the held-out blur by +0.144 for the from-scratch CNN and +0.217 for the ResNet-18 at blur of $\sigma = 0.6$, with all models trained on corrupted data performing better than the baseline. Additionally, adding the corruptions did not negatively impact the accuracy on clean images. Pretraining yields higher accuracy on unmodified images but it fails to improve robustness. This can be seen through how the ResNet-18 has a steeper decrease in performance when subjected to noise and blur conditions than the 20x smaller CNN. Blurred fakes have a persistently high ROC-AUC whilst recall drops significantly meaning that the fakes continue to be ranked higher than real images, so the failure is connected to the decision threshold, as opposed to losing the signal. Corruption augmentation is close to being free so it should be a default, but a pretrained model is not a suitable approach for ensuring robustness.**

Keywords. AI generated image detection, CIFAKE, data augmentation, transfer learning, corruption robustness.

I. Introduction

Generative AI models have made it extremely easy to generate fake images, and as a result, seeing is no longer believing. The CIFAKE dataset uses Stable Diffusion 1.4 to encapsulate the fabricated half of the dataset because models such as these are able to produce images which could easily be mistaken for real photographs [1]. The optimal approach for this dataset has been documented as correctly classifying images with a 92.98% accuracy [1], however this has been earned specifically on images which have not been tampered with. Therefore this approach is not entirely reflective of real-world synthetic images which have more often than not been passed into a messaging app or social media platform, resulting in JPEG compression, resizing and the noise associated with taking a screenshot of a screenshot. These corruptions are the result of the inherent infrastructure of the apps they are passed through, not necessarily driven by malintent, and could mean that these corrupted images no longer have the subtle pixel level traces that a detector may rely on to make a classification. If a detector's capabilities are limited to analysing clean data, then it is not built to make a pragmatic impact, and prior work demonstrates this exact fragility. Performance has been shown to drop sharply when test images are compressed or resized [2]. This is important when the ultimate goal of these models is to reinstate the ability to discern truth from fabrication, but they are not functional under the circumstances where this objective is most pressing.

Our project explores the question of model robustness on a CIFAR scale. A 32x32 benchmark cannot define what a production detector would score, but it is able to answer questions about how we should *design* these detectors regarding the transfer of augmentation, the impact of pre-training and the impact of robustness on clean accuracy. This project also enables us to use multiple seeds and exact controls. This would not be affordable for us to apply to a full scale model but could help to inform the design of a production model.

This exploration into the impact of data augmentation on image classification is well established, and we have adapted Hendrycks and Dietterich's approach to use as a benchmark [3]. Their protocol evaluates ImageNet and CIFAR classifiers performance on data which has been systematically corrupted at different severity levels, and their findings suggest that networks experience a decrease in accuracy when they are subjected to corruption, even if these corruptions are not noticeable to humans. The method they use is based on the idea that the robustness of a model must be measured using corruptions that the model has not been fine-tuned on, so we have applied this by keeping one corruption family out of our training process completely. In terms of training for fake-image detection, Wang et al. find that when a ResNet-50 is trained on ProGAN images with JPEG and blur augmentation, it is surprisingly good at generalising to unseen generators [4] however rather than asking how training on augmented

images impacts a model's learning, this project explores whether the impact of this learning is able to generalise to augmentation types that the training data did not contain. Corvi et al. [2] study the detection of diffusion-generated images and come to the conclusion that the discriminative signal mostly comes from the forensic traces that compression and resizing make weaker. This gives us a mechanism to test because if detection is dependent on high frequency traces, blur should cause the fake images to be missed rather than the real images to be incorrectly flagged. Another paper by Hendrycks et al. also explores how pretraining itself can improve robustness to corruption [5] on CIFAR-10-C, achieving a much lower corruption error for pretrained networks than networks built from scratch. All together, these strands of reasoning contribute to our experiment design, whereby we train detectors (from scratch CNNs and pretrained networks) with and without any image corruptions whilst also keeping one type of corruption outside of the training process to use during testing. This allows us to interrogate the difference between how well a pretrained model versus a from-scratch-CNN are able to generalise across degradation types.

The CIFAKE [1] dataset is used which couples 60,000 CIFAR-10 real photographs [6] with 60,000 Stable Diffusion v1.4 counterparts, whereby the fakes generated match the 10 categories associated with the CIFAR-10 dataset, ensuring that the two halves of the dataset depict the same things. We chose CIFAKE deliberately because the classes are perfectly balanced so accuracy can easily be read using a 0.5 baseline, meanwhile the train versus test split of 100,000 and 20,000 is large and stable for making estimations. Additionally the small resolution of 32x32 makes it very affordable to do the 12 seeded training runs we apply, allowing us to analyse every impact against a multi-seed spread, making our results more informative than a single seed run. The fact that the dataset consists of CIFAR-10 for the real half of the images means that we are able to reasonably take the exact corruption severities used in the CIFAR-10-C generation code [3], saving us from having to come up with our own levels of augmentation.

Over the course of three paired seeds, the 544k parameter CNN and the ImageNet-pretrained ResNet-18 [7, 8] perform 2 different training strategies, one using clean data and the other using augmentations of Gaussian noise and JPEG compression. These 12 runs are evaluated on test images put under 16 conditions: clean data as well as 5 different severities each for noise, JPEG compression and Gaussian blur. Blur is held out of training so the rows where the models' performances on blur are measured act as a scale for generalised robustness. Each condition is scored using accuracy, precision, F1, recall and ROC-AUC in order to explore evaluation beyond accuracy. These measures as well as the use of the pretrained architecture are incorporated in accordance with the feedback provided on our project proposal.

We began the project with 3 different hypotheses, the first one being that robustness would cost clean accuracy as we expected the baseline to achieve better on unmodified images. The second hypothesis was that we expected the augmentation would help the most for the corruptions the model has been trained on and the transferred benefit on the held out blur would be less. Our final hypothesis was that the pretrained network would be more robust as discussed by Hendrycks et al. [5]. Our experimentation results aligned with the idea that the augmentation would help with noise and JPEG compression, however the transferred benefit on blur was much larger than anticipated. Our other two hypotheses were refuted entirely because the robust models were a small amount better on clean data and the ResNet turned out to be more accurate on clean data but more fragile when facing corrupted images than the much smaller model was.

This report has three findings. Corruption augmentation transfers to a corruption family which has not been seen in training, improving the held-out blur accuracy by +0.144 for the from-scratch CNN and +0.217 for the ResNet-18 at $\sigma = 0.6$, having no negative impact on clean image detection. ImageNet pretraining improves clean accuracy but not robustness, with the un-augmented ResNet-18 falling further under noise and blur than the smaller CNN (although, our experiment design alters both the architecture and pretraining together so they cannot be separated from one another entirely). Finally, the metrics beyond accuracy highlight that the detectors fail through recall when blur is applied while the ROC-AUC remains high, so accuracy in isolation conceals a detector that has almost entirely stopped detecting.

II. Methodology and Implementation

A. Dataset

CIFAKE [1] pairs 60,000 CIFAR-10 images [6] with 60,000 Stable Diffusion v1.4 generated fake counterparts, split into 100,000 training and 20,000 test images with all of the classes being balanced. The fake images are assigned the positive class of label 1, so precision, recall and F1 are in relation to detecting fakes. To do this, we flipped the label order assigned by torchvision's ImageFolder. The notebook then checks a sample of test images to ensure this flip worked because an inverted label order would not raise an error even if the results were wrong. For the sake of mitigating a download failure, the loader also counts up every disk file (50,000 per class in train and 10,000 per class in test) so it can be stopped before training wastes compute on the wrong data.

B. Corruption protocol

All trained models are evaluated using the same 16 conditions of clean images and the 3 different corruption types (Gaussian noise, JPEG compression and Gaussian blur), each one consisting of 5 different severity levels. The severity grids are taken exactly from the 32x32 version of Hendrycks and Dietterich's make_cifar_c.py [3], which also concurs with the published source. The noise levels are $\sigma \in \{0.04, 0.06, 0.08, 0.09, 0.10\}$ added per pixel and clamped to [0,1]. Blur uses $\sigma \in \{0.4, 0.6, 0.7, 0.8, 1.0\}$ with a kernel size of $2\lceil 3\sigma \rceil + 1$. JPEG re-encodes each image at quality $\in \{80, 65, 58, 50, 40\}$. These corruptions are done to raw [0,1] tensors prior to channel normalisation in order to match how a real world image would arrive to the detector already corrupted, and therefore in advance of the channels getting normalised. Fig. 1 displays an example of a test image subjected to each augmentation type.

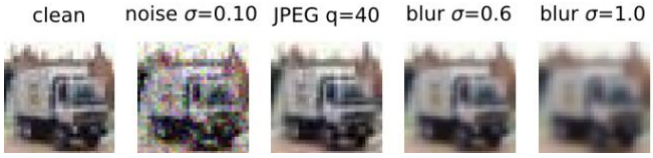

Fig. 1. A single CIFAKE image under the different evaluation conditions and all produced by the same corrupt() function. Blur σ=0.6 removes most of the pixel level texture at 32x32.

The robust training strategy augments only with noise and JPEG with blur held out of training for all models. Our proposal originally intended for us to augment using blur and noise before testing on the same corruptions, but this would only display how the model is able to memorise the augmentation type. Keeping one family of corruptions outside of the training process leaves us with a corruption that has not been witnessed by the model, so any robust advantage is generalisation as opposed to the memorisation of the training data. The notebook also asserts that the held out set {blur} and the augmented set {noise, JPEG} never overlap.

C. Architectures

The BaselineCNN consists of two conv-pool blocks and two fully connected layers, resulting in 544,066 parameters. The 32x32 input images mean that the conv blocks are 64x8x8 tensors, meaning that the first linear layer sees 4096 features, whilst outputting logits for the classes and training using cross-entropy.

Torchvision's ResNet-18 [7, 9] is an already existing model and relies on ImageNet weights [8]. We fine tune this network. Due to the fact that the stock stem (7x7 stride-2 convolution then maxpooling) would reduce a 32x32 image by a factor of 4 before the first residual block, the stem used by He et al. for their CIFAR-10 network [7] replaces this instead. A 3x3 stride-1 convolution without maxpooling as well as an additional 2 class layer means that there are 11,169,858 parameters. However, this change means that the pretrained 7x7 stem weights are removed and the first layer therefore trains from scratch. We also normalise using the CIFAR-10 channel statistics, not the ImageNet statistics associated with the remaining pretrained weights in the ResNet. These are costs that we have decided to accept and consider in the analysis section.

D. Training and experimental design

All 12 of the runs use the same optimiser of Adam [10], weight decay 0 and batch size 128, the same schedule of maximum 30 epochs with patience set to 5, and the same rule that the restored weights will be the ones associated with the epoch where the clean validation loss was at its lowest. The only exception is the learning rate, and this was a purposeful discrepancy. The CNN trains from scratch at 10^{-3} whilst the ResNet-18 trains at 10^{-4} because at the start of training, updates at 10^{-3} will not allow the new layer to learn anything useful before overwriting the pretrained features. Both of these approaches validate on clean data even if the training data used is augmented in order to ensure the early stopping signal is the same for all strategies. See Table I for the entire training approach.

The robust strategy corrupts each training batch as it goes. Images have a 50% chance of being flipped horizontally to create even more data which are similar to the other images in the dataset (we could not flip vertically because many things upside down would be unrealistic and not fit into the dataset). The batch is separated into 8 different chunks, each one having a different corruption kind chosen from noise and JPEG compression, as well as one severity taken from that corruption type's grid. This ensures that every batch has a mixture of clean and heavily corrupted images. The baseline strategy only trains on clean images, not considering augmentation (not even flips) so that the comparison between the strategies can focus on the entirety of the augmentation process and not just one part of it.

TABLE I

Experiment matrix and training recipe. The 12 runs cross two architectures, two strategies and three seeds. The two strategies differ in the augmentation flag and in nothing else, asserted in code.

Architectures	BaselineCNN (544,066 params) ResNet-18, ImageNet-pretrained (11,169,858 params)
Strategies	baseline (clean training) robust (trained with noise + JPEG augmentation)
Seeds	42, 43, 44, paired across strategies
Optimiser	Adam, weight decay 0, batch size 128
Learning rate	10^{-3} (CNN); 10^{-4} (ResNet-18, fine-tuning)
Schedule	max. 30 epochs, early stopping with patience 5
Model selection	lowest clean validation loss, best weights restored
Validation split	90/10 of the training set, seeded per run
Augmentation	flip p=0.5; batch split into 8 chunks, each corrupted with a random kind and severity from the noise/JPEG grids
Held out	Gaussian blur, never used in any training run
Evaluation	16 conditions × 5 metrics on the 20,000-image test split

Three seeds of 42, 43 and 44 are run and the design is paired. For each seed both the baseline and the robust runs have the same 90 to 10 train-validation split and they have the same initial weights. This means that each difference between the robust and the baseline is a paired comparison. Paired runs were chosen over unpaired because with only 3 draws per configuration, cancelling the issues of split luck and initialisation luck results in more statistical reliability than adding another independent seed. Three seeds are not enough for a proper significance test so instead we report the mean ± sample standard deviation everywhere and check that all effects which have been claimed do in fact exceed the between seed spread.

Before starting any training measures, a smoke test was run which carried out 1 epoch of the CNN on 512 randomly selected images, succeeded with an evaluation using 16 conditions to assert finite losses and 16 result rows. It is important that the images are selected randomly because the CIFAKE folders put all of the fakes at the start, meaning that just taking the first 512 instances would result in all of the datapoints used in the smoke test being from the same class, making ROC-AUC undefined. Doing this allowed us to catch any mistakes in the pipeline before beginning the long process of training the models.

E. Metrics

Each run receives a score for all 16 conditions with 5 metrics: accuracy, precision, recall, F1 (fake as the positive class) and ROC-AUC. This has been done to address the feedback for our proposal by interrogating

metrics beyond accuracy. Recall refers to the fraction of fakes detected, precision refers to the level of trust that can be assigned to a fake verdict, F1 balances both precision and recall, and ROC-AUC measures if the model's scores rank fakes above reals regardless of the decision threshold. The project's output consists of 192 metric rows because there are 2 architectures, 2 strategies, 3 seeds and 16 conditions.

F. *Reproducibility*

Each run gets frozen as a three-file bundle consisting of a checkpoint, a JSON training history file (loss curves, most successful epoch, merged configuration used, dataset it was trained on and wall time), and a 16 condition CSV full of metrics. The analysis notebook never retrains and accepts a bundle after a check that all of the files exist, the configuration matches the frozen protocol, the provenance of the dataset is a recognised CIFAKE release, the CSV has all 16 of the conditions in the protocol and the checkpoint loads under torch.load(weights_only=True), rejecting the pickled code and confirming the weights fit the architecture. The 2 CSV files are checked against the per run CSV files before any table is produced. A full run of the notebook validates all 12 bundles and regenerates everything in the report. Randomness is rigidly controlled throughout as every run has seeds for Python, NumPy and PyTorch, whilst cuDNN is restricted to deterministic kernels, which comes with the price of some speed but is worth it to make the runs repeatable. The analysis notebook regenerates every figure from the frozen bundles at full size, so Group_Neural_Networks_Cifake.ipynb is the reference for detailed inspection of the curves.

III. Experimentation and Evaluation

A. *Runs and Hardware*

We completed all 12 training runs in the experimental matrix. The matrix combines the BaselineCNN and pretrained ResNet-18, the baseline and robust training strategies, and the paired seeds 42, 43, and 44. Every completed run was then evaluated on the same 20,000-image test split under all 16 conditions. This produced 192 result rows, with five metrics recorded in each row. We ran the complete matrix rather than selecting the most favourable seed because the spread across three paired seeds shows whether a result survives changes in the split and initial weights.

The BaselineCNN runs trained locally on an Apple-silicon GPU via PyTorch's MPS backend. Individual runs took 85.5 to 264.7 seconds, for 17.2 minutes across the six runs. The pretrained ResNet-18 required more compute, so its six runs trained on a Colab T4 and took 678.8 to 1,301.7 seconds each, or 95.6 minutes in total. We didn't use training time to compare the architectures because they ran on different hardware. Every baseline-to-robust comparison within an architecture did use the same device, so the hardware difference does not explain the robustness gaps reported below.

Each trained checkpoint made 16 passes over the 20,000-image test split, and JPEG compression re-encoded each image on the CPU. We froze each completed run as a bundle that the analysis notebook checks and reuses to reproduce the tables and figures without retraining.

B. *Training Behaviour*

The baseline runs fitted the clean training data quickly and then overfitted. Across seeds 42 to 44, the BaselineCNN baseline stopped after 9 to 10 epochs at lr = 0.001, with its lowest clean validation loss at epoch 4 or 5. The pretrained ResNet-18 baseline stopped after 8 to 9 epochs at lr = 0.0001, with its lowest clean validation loss at epoch 3 or 4. Training loss continued to fall after those points while clean validation loss rose. We therefore restored the earlier weights selected by validation loss rather than keeping the final epoch.

The robust training strategy delayed early stopping under the same patience of 5 epochs. The robust BaselineCNN ran for 17 to 19 epochs at lr = 0.001 and selected epoch 12 or 14. The robust pretrained ResNet-18 ran for 11 to 14 epochs at lr = 0.0001 and selected epoch 6 or 9. The changing mixture of clean, noisy, and JPEG-compressed training images made memorising the training set harder. In the terms introduced in the week 3 material, the augmentation acted as a regulariser rather than merely teaching the models two corruption types.

Fig. 2 shows the seed 42 loss curves. The histories for the other seeds also place the lowest validation loss before the final epoch. The pretrained ResNet-18 reached lower clean validation losses than the BaselineCNN. Its best values across all six runs were 0.069 to 0.088, while the BaselineCNN reached 0.127 to 0.144. This advantage agrees with the training recipe in Table I and the clean test accuracy in Table II, but it does not establish corruption robustness. The later severity results separate those two outcomes.

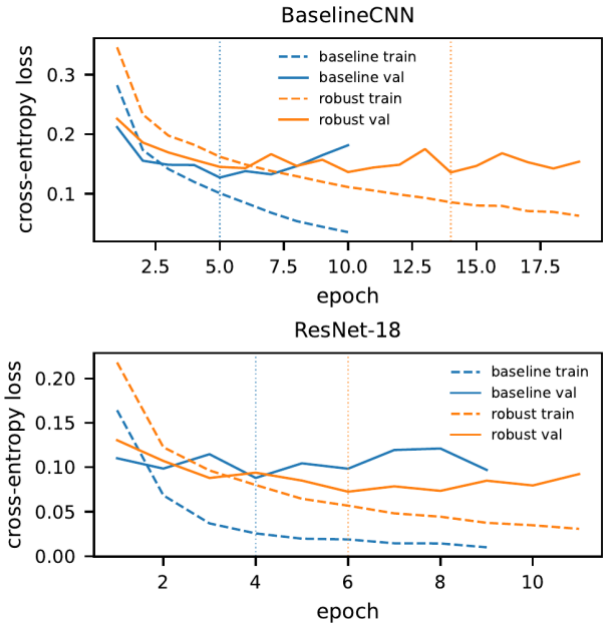

Fig. 2. Seed 42 training and validation loss curves. Dotted vertical lines mark the restored epoch selected by the lowest clean validation loss.

C. *Results across all conditions*

Table II reports mean test accuracy and sample standard deviation across seeds 42 to 44. Fig. 3 plots the same results against corruption severity. The clean baseline BaselineCNN reached 0.944 ± 0.001 against the 0.5 random baseline for two balanced classes. This starting point matters because the corruption comparison tests an already accurate detector

rather than repairing a deliberately weak one. The pretrained ResNet-18 started higher at 0.969 ± 0.002, but that 0.025 clean advantage did not continue under every corruption.

The robust training strategy protected both models against Gaussian noise. Across the five noise levels, robust BaselineCNN accuracy stayed between 0.934 and 0.947, while its baseline fell from 0.895 at σ = 0.04 to 0.803 at σ = 0.10. The robust pretrained ResNet-18 stayed between 0.962 and 0.971, while its baseline fell from 0.911 to 0.689. At σ = 0.10, robust training added 0.131 accuracy to the BaselineCNN and 0.272 to the pretrained ResNet-18.

JPEG compression caused smaller losses than noise or blur. At the harshest setting of q = 40, the baseline and robust BaselineCNN reached 0.883 and 0.928. The corresponding pretrained ResNet-18 results were 0.886 and 0.958. Robust training therefore added 0.045 and 0.072 accuracy at q = 40. The pretrained ResNet-18 retained a small absolute lead over the BaselineCNN throughout the JPEG grid, unlike its behaviour under severe noise and blur.

Gaussian blur provides the stricter test because no model encountered it during training. At σ = 0.4, all four models remained between 0.931 and 0.970. Accuracy then dropped sharply at σ = 0.6. The BaselineCNN fell from 0.931 to 0.693 and the pretrained ResNet-18 fell from 0.955 to 0.657. Robust training raised the σ = 0.6 results to 0.837 for the BaselineCNN and 0.874 for the pretrained ResNet-18. These gains of 0.144 and 0.217 cannot be memorisation of blur because the robust training strategy used only noise and JPEG compression.

Robustness did not cost clean accuracy in these runs. The robust BaselineCNN reached 0.949 ± 0.001 against 0.944 ± 0.001 for its baseline. The robust pretrained ResNet-18 reached 0.973 ± 0.003 against 0.969 ± 0.002. These gains of 0.005 and 0.004 are small, so we treat the result as no clean cost rather than a meaningful clean improvement. This finding rejects our original expectation that robustness would trade against performance on clean images.

The sample standard deviations also distinguish stable improvements from seed-specific ones. On noise, the baseline pretrained ResNet-18 reached standard deviations as high as 0.029, while the robust version stayed between 0.003 and 0.004. Blur produced the largest spread for both strategies, reaching 0.037 for the robust pretrained ResNet-18 at σ = 0.8. We therefore trust the direction of the held-out improvement more than its exact size. All reported robust means remain above their matching baselines.

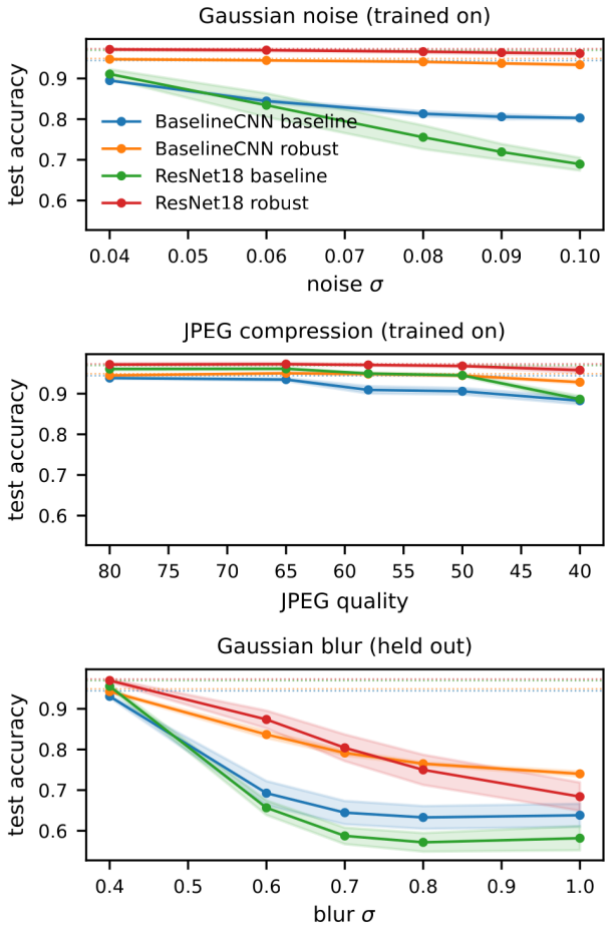

Fig. 3. Test accuracy against corruption severity, shown as the mean across seeds 42 to 44 with sample standard deviation bands. We hold Gaussian blur out of training.

TABLE II

Test accuracy under all 16 conditions, mean ± sample SD over seeds 42 to 44. No training batch contains Gaussian blur.

Condition	Baseline CNN baseline	Baseline CNN robust	ResNet-18 baseline	ResNet-18 robust
clean	0.944 ± 0.001	0.949 ± 0.001	0.969 ± 0.002	0.973 ± 0.003
noise σ = 0.04	0.895 ± 0.000	0.947 ± 0.001	0.911 ± 0.012	0.971 ± 0.003
noise σ = 0.06	0.844 ± 0.004	0.944 ± 0.001	0.834 ± 0.029	0.970 ± 0.003
noise σ = 0.08	0.813 ± 0.006	0.941 ± 0.002	0.755 ± 0.029	0.966 ± 0.003
noise σ = 0.09	0.806 ± 0.006	0.937 ± 0.001	0.719 ± 0.020	0.964 ± 0.004
noise σ = 0.10	0.803 ± 0.003	0.934 ± 0.003	0.689 ± 0.016	0.962 ± 0.004
JPEG q = 80	0.938 ± 0.002	0.945 ± 0.001	0.961 ± 0.003	0.972 ± 0.003
JPEG q = 65	0.935 ± 0.004	0.950 ± 0.001	0.961 ± 0.002	0.973 ± 0.003
JPEG q = 58	0.909 ± 0.009	0.948 ± 0.003	0.949 ± 0.002	0.971 ± 0.003
JPEG q = 50	0.906 ± 0.008	0.945 ± 0.002	0.945 ± 0.003	0.968 ± 0.003
JPEG q = 40	0.883 ± 0.009	0.928 ± 0.004	0.886 ± 0.008	0.958 ± 0.006
blur σ = 0.4, held out	0.931 ± 0.003	0.942 ± 0.003	0.955 ± 0.006	0.970 ± 0.004
blur σ = 0.6, held out	0.693 ± 0.030	0.837 ± 0.009	0.657 ± 0.018	0.874 ± 0.022

blur σ = 0.7, held out	0.645 ± 0.029	0.791 ± 0.007	0.587 ± 0.020	0.804 ± 0.033
blur σ = 0.8, held out	0.633 ± 0.028	0.765 ± 0.004	0.572 ± 0.023	0.750 ± 0.037
blur σ = 1.0, held out	0.638 ± 0.028	0.740 ± 0.006	0.582 ± 0.030	0.684 ± 0.035

D. Beyond Accuracy

Accuracy alone hides how blur changes the detectors' decisions. At blur σ = 1.0, the baseline pretrained ResNet-18 retained precision of 0.951 but recall fell to 0.174, leaving F1 at 0.289. The baseline BaselineCNN showed the same direction, with precision of 0.934, recall of 0.301, and F1 of 0.450. These models rarely labelled a real image as fake, but they missed most fake images. For an AI-generated image detector, this is a more serious failure than the corresponding accuracies of 0.582 and 0.638 suggest.

ROC-AUC shows whether the problem lies only at the decision threshold or also in the learned ranking. At blur σ = 0.6, the baseline pretrained ResNet-18 achieved ROC-AUC of 0.948 despite accuracy of 0.657 and recall of 0.314. The ROC-AUC shows that the scores still ranked fake images above real images, but blur made the clean-data threshold misclassify them. The robust training strategy improved the same condition to 0.987 ROC-AUC, 0.754 recall, and 0.856 F1.

At blur σ = 1.0, the baseline ranking itself weakened. ROC-AUC fell to 0.811 for the BaselineCNN and 0.787 for the pretrained ResNet-18. The robust versions retained 0.896 and 0.916. A threshold adjustment could help where ROC-AUC remains high, as it does at σ = 0.6, but it cannot fully repair a representation whose ranking has deteriorated. This distinction is why we report all five metrics rather than treating accuracy as a complete evaluation.

Seed 44 also shows why averages need care under severe noise. At noise σ = 0.10, the three baseline pretrained ResNet-18 runs reached accuracies of 0.676, 0.707, and 0.685. Seeds 42 and 43 mainly missed fakes, with recalls of 0.561 and 0.488. Seed 44 instead reached recall of 0.909 but precision of 0.628, so it labelled many more real images as fake. The mean precision of 0.741 and mean recall of 0.653 describe no individual run. The paired three-seed design exposes that change in failure mode, while one selected run would conceal it.

Table III keeps the metric comparison to five headline conditions per model. The clean row anchors each model, noise σ = 0.10 and JPEG q = 40 are the harshest trained-on conditions, and blur σ = 0.6 and σ = 1.0 show moderate and severe held-out corruption. The robust training strategy improves accuracy, F1, and ROC-AUC in every listed corrupted condition.

TABLE III

Headline test conditions with metrics beyond accuracy, using three-seed means. F1 treats fakes as the positive class. The accuracy gap is robust minus baseline.

Model	Condition	Accuracy baseline	Accuracy robust	Accuracy gap	ROC-AUC baseline	ROC-AUC robust	F1 baseline	F1 robust
BaselineCNN	clean	0.944	0.949	+0.005	0.988	0.989	0.944	0.949
BaselineCNN	noise σ = 0.10	0.834	0.965	+0.131	0.904	0.983	0.774	0.934
BaselineCNN	JPEG q = 40	0.828	0.873	+0.045	0.974	0.982	0.870	0.926
BaselineCNN	blur σ = 0.6	0.637	0.781	+0.144	0.919	0.961	0.558	0.809
BaselineCNN	blur σ = 1.0	0.640	0.702	+0.102	0.811	0.896	0.450	0.660
ResNet-18	clean	0.969	0.973	+0.004	0.996	0.997	0.969	0.973
ResNet-18	noise σ = 0.10	0.689	0.962	+0.272	0.806	0.994	0.667	0.961
ResNet-18	JPEG q = 40	0.858	0.930	+0.072	0.985	0.994	0.873	0.957
ResNet-18	blur σ = 0.6	0.657	0.874	+0.217	0.948	0.987	0.477	0.856
ResNet-18	blur σ = 1.0	0.582	0.684	+0.102	0.787	0.916	0.289	0.543

IV. Analysis and Conclusion

A. The three hypotheses tested against seed noise

1) Hypothesis 1: robustness would cost clean accuracy

Our first hypothesis was that "robustness would cost clean accuracy as we expected the baseline to achieve better on unmodified images." The results refute it for both architectures. On clean test images, the baseline BaselineCNN reached 0.944 ± 0.001 accuracy, while the robust BaselineCNN reached 0.949 ± 0.001. The corresponding pretrained ResNet-18 results were 0.969 ± 0.002 and 0.973 ± 0.003. Robust training therefore changed clean accuracy by +0.005 for the BaselineCNN and +0.004 for the pretrained ResNet-18. These small differences show no clean accuracy cost, but we didn't claim that corruption augmentation meaningfully improves clean detection.

The paired design supports that verdict more directly than the rounded means alone. The robust-minus-baseline clean accuracy gap was positive for all three seeds in both architectures. For the BaselineCNN, the three paired gaps ranged from +0.004 to +0.006. For the pretrained ResNet-18, they ranged from +0.003 to +0.005. This consistency matters because a mean close to zero could otherwise hide opposing seed effects. Our result instead shows the same small direction under every paired split and initialisation. Within this experiment, exposing the models to Gaussian noise and JPEG compression broadened the training distribution without sacrificing their ability to classify clean CIFAKE images.

2) Hypothesis 2: the largest benefit would stay with trained-on corruptions

Our second hypothesis was that "we expected the augmentation would help the most for the corruptions the model has been trained on and the transferred benefit on the held out blur would be less." The results support the first clause but refute the expected limit on transfer. For the BaselineCNN and ResNet-18 respectively, robust training improved accuracy by +0.131 and +0.272 at noise σ = 0.10, and by +0.045 and +0.072 at JPEG q = 40. The smaller JPEG gaps reflect the fact that both baseline models remained comparatively accurate under compression, reaching 0.883 and 0.886 at q = 40.

The held-out result was not weaker in the way we expected. At blur σ = 0.6, robust training improved the BaselineCNN by +0.144 and the pretrained ResNet-18 by

+0.217. The BaselineCNN blur gain was larger than its gain at the harshest noise level and more than three times its gain at the harshest JPEG level. The pretrained ResNet-18 blur gain was smaller than its exceptional +0.272 noise gain, but it was still three times its +0.072 JPEG gain. Averaged across all five blur severities, robust training added 0.107 accuracy to the BaselineCNN and 0.146 to the pretrained ResNet-18. Every mean robust-minus-baseline gap across the 16 conditions was positive for both architectures. The effect is therefore not confined to one selected blur severity or one favourable seed.

3) Hypothesis 3: the pretrained network would be more robust

Our final hypothesis was that "the pretrained network would be more robust as discussed by Hendrycks et al. [5]." The clean results initially appear to support it. The baseline pretrained ResNet-18 reached 0.969 accuracy against 0.944 for the baseline BaselineCNN, a +0.025 advantage. Under corruption, however, the ranking depended on the condition. At JPEG q = 40, the two baseline models were nearly equal at 0.886 and 0.883. At noise $\sigma = 0.10$, the pretrained ResNet-18 fell to 0.689 while the BaselineCNN retained 0.803. At held-out blur $\sigma = 0.6$, they reached 0.657 and 0.693, and at $\sigma = 1.0$ they reached 0.582 and 0.638. The pretrained network therefore lost its clean advantage and became 0.114 less accurate under severe noise, 0.036 less accurate under moderate blur, and 0.056 less accurate under severe blur.

Our results refute this hypothesis. ImageNet pretraining bought clean accuracy, but the unaugmented pretrained ResNet-18 was not more robust than the from-scratch BaselineCNN. Robust training lifted the pretrained network to 0.962 at noise $\sigma = 0.10$ and 0.874 at blur $\sigma = 0.6$. This means that pretraining did not prevent robustness from being learned. It simply did not provide that robustness by itself.

B. Held-out blur transfer

Corruption augmentation transferred to blur, which no model encountered during training. We excluded blur from robust training and varied noise and JPEG compression within training batches. Any improvement on blur therefore measures transfer across degradation types rather than direct practice on the test corruption. At $\sigma = 0.6$, that transfer raised BaselineCNN accuracy from 0.693 to 0.837 and pretrained ResNet-18 accuracy from 0.657 to 0.874. The paired gains were positive for every seed. They ranged from +0.102 to +0.177 for the BaselineCNN and from +0.187 to +0.233 for the pretrained ResNet-18.

The transfer continued at stronger blur, although neither robust model became invariant to it. At $\sigma = 1.0$, robust training raised the BaselineCNN from 0.638 to 0.740 and the pretrained ResNet-18 from 0.582 to 0.684. Both gains were about +0.102, but Fig. 3 shows that robust training only shifts the blur curves upwards. Both final accuracies remained far below their clean results as blur strengthened.

Robust training repaired the threshold failure that Section III identified at blur $\sigma = 0.6$, improving both ranking and the recovery of fake images. At $\sigma = 1.0$, the baseline pretrained ResNet-18 fell to 0.787 ROC-AUC and 0.174 recall, while the robust version retained 0.916 ROC-AUC and 0.380 recall. The remaining recall shortfall marks the limit of the transfer.

These results support the high-frequency-traces mechanism proposed in the Introduction [2], [4]. If a detector depends on fragile high-frequency traces, smoothing should make fake images harder to detect. Section III showed both baselines pairing high precision with collapsed recall at severe blur. The models became conservative rather than randomly confused. Noise and JPEG training improved held-out blur performance at every tested severity, which suggests that the robust models depended less on one narrow set of traces.

C. What pretraining did and did not buy

Our finding does not contradict every published result that associates pretraining with corruption robustness. It shows that the benefit did not transfer automatically under our particular architecture, data, and fine-tuning choices. We changed the torchvision ResNet-18 stem for 32×32 inputs by replacing the original large, strided input operation with a 3×3 stride-1 convolution and removing max-pooling [7]. We replaced the stem because the stock operation would shrink the input before the first residual block. Our network therefore differs from the unchanged ImageNet model used in some robustness studies. We newly initialised the earliest operations, so they did not inherit ImageNet features for local texture and degradation.

We also fine-tuned all layers on CIFAKE rather than freezing a pretrained representation or evaluating it without task-specific training. The pretrained ResNet-18 used lr = 0.0001 and selected weights using clean validation loss. Full fine-tuning can adapt useful ImageNet features to the clean real-versus-fake boundary while also specialising them to CIFAKE's Stable Diffusion traces. Clean validation selection provides no direct incentive to preserve performance under noise or blur. Our experiment changes architecture and pretraining together as well, because the BaselineCNN and ResNet-18 differ in depth, capacity, residual connections, and initialisation. We can conclude that this pretrained ResNet-18 configuration did not outperform this from-scratch CNN under severe noise and blur. We cannot isolate pretraining as the sole cause of the difference.

D. Limitations

Three paired seeds expose direction and spread, but they do not support a formal significance test. The blur gains exceeded the between-seed spread at the central $\sigma = 0.6$ condition, yet their exact sizes remain uncertain. The largest seed spreads, such as the robust pretrained ResNet-18 under strong blur, are large enough to discourage fine-grained ranking between nearby configurations. More seeds would narrow the uncertainty and test whether every paired effect persists.

We held out one corruption family, Gaussian blur. Keeping it outside training gives the transfer claim a clean internal test, but it does not establish transfer to resizing, colour shifts, contrast changes, screenshots, social-media processing chains, or combinations of corruptions. CIFAKE contributes a further limit because its fake half comes from one generator family and both classes use 32×32 images. A detector may learn traces specific to Stable Diffusion 1.4 [11] or to the construction of this dataset. Our results cannot establish robustness to later generators, higher resolutions, or real deployment data.

Finally, the BaselineCNN and pretrained ResNet-18 trained on different hardware. Comparisons of accuracy remain valid because each paired baseline-to-robust comparison used the same device and evaluation protocol. Runtime comparisons across architectures do not. The mixed hardware also prevented a controlled comparison of

computational efficiency, energy use, or robustness gained per unit of training cost.

E. Future work

The next experiment should retain the paired design while adding more seeds and rotating the held-out corruption family. Training separate models with blur, noise, JPEG compression, resizing, and contrast changes held out in turn would test whether transfer is general or unusually favourable for blur. Evaluating compound corruptions would then better represent images that have been resized, compressed, and captured again rather than degraded in only one way.

A second extension should separate architecture from pretraining. We would compare random and ImageNet initialisation in the same ResNet-18. We would then test the stock stem against the replacement stem. Freezing early layers before staged fine-tuning would test whether full CIFAKE adaptation erases useful pretrained robustness. Model selection could also include a corruption validation split while preserving a final untouched test set.

An external test should move beyond CIFAKE. Cross-generator evaluation should train on Stable Diffusion 1.4 and test on unseen diffusion and adversarial generators, alongside real images from a different source. A threshold sweep should measure how much recall can be recovered at a fixed false-positive rate under moderate blur. None of these extensions should replace the five-metric evaluation, since recall and ROC-AUC reveal failures that clean accuracy conceals.

F. Conclusion

Across 12 paired runs, robust training improved every mean test condition for both the BaselineCNN and pretrained ResNet-18 without reducing clean accuracy. The effect transferred beyond the Gaussian noise and JPEG compression used in training. At held-out blur $\sigma = 0.6$, accuracy rose by +0.144 for the BaselineCNN and +0.217 for the pretrained ResNet-18. ImageNet pretraining raised clean accuracy by 0.025 in the baseline comparison, but it didn't provide automatic robustness to severe noise or blur. The multi-metric results explain the practical failure, with blur causing fake-image recall to collapse even when precision and ROC-AUC remained comparatively high. Three seeds don't support a significance claim. One held-out corruption family and one CIFAR-scale generator dataset also limit the scope of the result. These limits do not change the practical result. We should test degradation transfer directly because clean accuracy does not measure it. The answer to the question we set at the start is yes for augmentation and no for pretraining: corrupting training batches produced detectors that keep working on degraded images, while ImageNet weights alone did not.

REFERENCES

[1] J. J. Bird and A. Lotfi, "CIFAKE: Image classification and explainable identification of AI-generated synthetic images," IEEE Access, vol. 12, pp. 15 642–15 650, 2024.

[2] R. Corvi, D. Cozzolino, G. Zingarini, G. Poggi, K. Nagano, and L. Verdoliva, "On the detection of synthetic images generated by diffusion models," in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2023, pp. 1-5.

[3] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in Proc. Int. Conf. Learning Representations (ICLR), 2019.

[4] S.-Y. Wang, O. Wang, R. Zhang, A. Owens, and A. A. Efros, "CNN-generated images are surprisingly easy to spot... for now," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2020, pp. 8695–8704.

[5] D. Hendrycks, K. Lee, and M. Mazeika, "Using pre-training can improve model robustness and uncertainty," in Proc. Int. Conf. Machine Learning (ICML), 2019, pp. 2712–2721.

[6] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.

[8] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2009, pp. 248–255.

[9] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019, pp. 8024–8035.

[10] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. ICLR, 2015.

[11] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in Proc. IEEE/CVF CVPR, 2022, pp. 10684-10695.